

Finding a Longest Increasing Subsequence from the Paths in a Complete Bipartite Graph*

Guan-Yu Lin¹, Jia-Jie Liu², and Yue-Li Wang^{1,†}

¹ Department of Information Management,
National Taiwan University of Science and Technology, Taipei, Taiwan

² Department of Information Management,
Shih Hsin University, Taipei, Taiwan

Abstract

Let $S = s_1, s_2, \dots, s_n$ be an integer sequence. The longest increasing subsequence problem is to find an increasing subsequence of S with the longest length. By regarding S as a weight sequence of the vertices in a path, we can redefine the longest increasing subsequence problem on graphs as follows. Let $G = (V, E)$ be a graph in which every vertex $v \in V$ has a weight $w(v)$. A longest increasing subsequence of G is a longest increasing weight subsequence among all paths in G . Thus, the longest increasing subsequence problem on G is to find a longest increasing subsequence of G . In this paper, we present an $O(|V| \log |V|)$ -time algorithm for finding a longest increasing subsequence of complete bipartite graphs $K_{m,n}$ with $m \geq n$.

Keywords: Longest increasing subsequence; Complete bipartite graphs; Paths.

1. Introduction

Let $S = \langle s_1, s_2, \dots, s_n \rangle$ be an integer sequence. A subsequence of S is obtained by deleting zero or some (not necessarily consecutive) numbers from S . An increasing subsequence of S is a subsequence in which all numbers are in increasing order. A longest increasing subsequence of S is an increasing subsequence with the maximum length. By regarding S as a weight sequence of the vertices in a path, we can redefine the longest increasing subsequence problem on graphs as follows. Let $G = (V, E)$ be a graph in which every vertex $v \in V$ has a weight $w(v)$. An increasing weight subsequence of

path $\mathcal{P}$ is a subsequence of $\mathcal{P}$ whose vertex weights are in increasing order. The length of an increasing weight subsequence S in path $\mathcal{P}$ is denoted by $\ell_{\mathcal{P}}(S)$. A longest increasing weight subsequence (LIS for short) of $\mathcal{P}$ is an increasing weight subsequence of $\mathcal{P}$ with the longest length. A longest increasing subsequence of G is a longest increasing weight subsequence among all paths in G . The length of a longest increasing subsequence of G is denoted by $L(G)$.

The LIS problem was proposed by Schensted [14]. A direct solution to computing an LIS, running in $O(n \log n)$ time, was proposed in [10, 11, 14]. This solution is optimal if the elements are drawn from an arbitrary set of n numbers due to the $\Omega(n \log n)$ lower bound for sorting n elements. For the input sequence which is a permutation of $\{1, 2, \dots, n\}$, $O(n \log \log n)$ -time algorithms were proposed in [3, 13]. It relies on the priority search trees of van Emde Boas data structure [4, 5], which provide $O(\log \log n)$ amortized time per operation when keys are drawn from the set $\{1, 2, \dots, n\}$. Furthermore, Crochemore and Porat [8] improved the long-standing $O(n \log \log n)$ upper bound down to $O(n \log \log k)$, where k is the length of LIS. Bespamyatnikh and Segal [3] presented an $O(n \log \log n)$ time algorithm to enumerate all LIS. Aldous and Diaconis [2] and Groeneboom [12] showed the probabilistic result to average length of increasing subsequences in a random permutation.

There exist many kinds of variations of the LIS problem. Albert et al. [1] and Chen et al. [6] considered the problem of finding the longest increasing subsequence in a sliding window over a given sequence. Elmasry [9] proposed the longest almost-increasing subsequence problem that allows increasing subsequence to decrease in a fixed value. Yang, Huang, and Chao [15] solved the longest common increasing subsequence problem whose goal is to find a maximum-length common

*This research was partially supported by National Science Council of Taiwan under Contracts NSC100-2221-E-011-068- and NSC100-2221-E-011-067-MY3.

†All correspondence should be addressed to Professor Yue-Li Wang, Department of Information Management, National Taiwan University of Science and Technology, Taipei, Taiwan, R.O.C. (Email: ylwang@cs.ntust.edu.tw).

subsequence of the two sequences such that the subsequence is increasing. For the longest increasing subsequence problem on graphs, Chen et al. [7] proposed an $O(n \log \log n)$ -time and $O(n)$ -space algorithm for finding an LIS on galled trees, where n is the number of vertices in galled tree. In this paper, we propose an $O(n \log n)$ -time algorithm for finding an LIS on complete bipartite graphs $K_{m,n}$, where $m \geq n$.

The rest of this paper is organized as follows. In Section 2, we propose an algorithm for finding an LIS on complete bipartite graphs. Concluding remarks are given in Section 3.

2. Finding an LIS on complete bipartite graphs

A graph $G = (V, E)$ is called *bipartite* if its vertex set V can be partitioned into two sets $X = \{x_1, x_2, \dots, x_m\}$ and $Y = \{y_1, y_2, \dots, y_n\}$, i.e., $V = X \cup Y$, such that every edge has one end in X and the other end in Y . A bipartite graph is *complete*, denoted by $K_{m,n}$ if every vertex in X is joined to all vertices of Y . For each $v \in V$, let $w(v)$ denote the *weight* of vertex v . A *path* $\mathcal{P}$ of length $k - 1$ from vertex x to vertex y in $K_{m,n}$ is a sequence of distinct vertices $v_1, v_2, \dots, v_k$ with $x = v_1$, $y = v_k$, and $v_i v_{i+1} \in E(K_{m,n})$, for $i = 1, 2, \dots, k - 1$. The vertex just before (respectively, after) v_i in path $\mathcal{P}$ is denoted by v_i^- (respectively, v_i^+).

Let $\mathcal{P}$ be a path in $K_{m,n}$ and y_i and y_j with $i \neq j$ be two vertices in $\mathcal{P}$. For ease of description, we define some operations on $\mathcal{P}$ such that the resulting path has the same vertex set as $\mathcal{P}$ has. It is clear that swapping y_i and y_j in $\mathcal{P}$, denoted by $\mathcal{S}_{\mathcal{P}}(y_i, y_j)$, is still a path in $K_{m,n}$. The path obtained by moving a consecutive pair $y_j y_j^+$ (respectively, $y_j y_j^-$) from its original positions to just before y_i , denoted by $M_{\mathcal{P}}(y_j y_j^+, y_i)$ (respectively, $M_{\mathcal{P}}(y_j y_j^-, y_i)$), is also a path in $K_{m,n}$. Note that if $\mathcal{P}$ is the path

$v_1, \dots, y_i^-, y_i, y_i^+, \dots, (y_j^-)^-, y_j^-, y_j, y_j^+, (y_j^+)^+, \dots, v_p$,
then $M_{\mathcal{P}}(y_j y_j^+, y_i)$ is the path
 $v_1, \dots, y_i^-, y_j, y_j^+, y_i, y_i^+, \dots, (y_j^-)^-, y_j^-, (y_j^+)^+, \dots, v_p$,
and $M_{\mathcal{P}}(y_j y_j^-, y_i)$ is the path
 $v_1, \dots, y_i^-, y_j, y_j^-, y_i, y_i^+, \dots, (y_j^-)^-, y_j^+, (y_j^+)^+, \dots, v_p$.

For brevity, in the rest of this paper, we assume without loss of generality that X and Y are the partite sets of complete bipartite graph $K_{m,n}$ with $m \geq n$ and each vertex v in $K_{m,n}$ has a distinct weight $w(v)$, where $X = \{x_1, x_2, \dots, x_m\}$ and $Y = \{y_1, y_2, \dots, y_n\}$. Thus, the following proposition holds directly.

Lemma 1. *For a complete bipartite graph $K_{m,n}$*

with partite sets X and Y , there exists an LIS of $K_{m,n}$ containing all vertices in Y .

Proof. Let S be an LIS of $K_{m,n}$. If S also contains all vertices in Y , then the lemma follows directly. For the case where S does not contain all vertices in Y , we shall prove that there always exists another LIS S' of $K_{m,n}$ which has one more vertex in Y than S . This also implies that there exists an LIS of $K_{m,n}$ containing all vertices in Y . Assume that S is a subsequence of path $\mathcal{P} = v_1, v_2, \dots, v_p$. It is obvious that if $m > n$, then $p = 2n + 1$; otherwise, $p = 2n$. This means that all vertices in Y are also in $\mathcal{P}$. Assume that there exists a vertex y_o which is not in S . In the following, we show how to construct $\mathcal{P}'$ from $\mathcal{P}$ such that S' has one more vertex in Y than S and $\ell_{\mathcal{P}'}(S') = \ell_{\mathcal{P}}(S)$.

Let y_l and y_r be the two vertices in S with the largest and smallest indices, respectively, such that $w(y_l) < w(y_o)$ and $w(y_r) > w(y_o)$. It is clear that at least one of y_l and y_r must exist; otherwise, it is easy to find an increasing weight subsequence S' with $\ell_{\mathcal{P}}(S') = \ell_{\mathcal{P}}(S) + 1$, a contradiction. By taking into consideration the existence of y_l and y_r , we have three subcases to consider which are

- (1) y_l does not exist,
- (2) y_r does not exist, and
- (3) both y_l and y_r exist.

We only discuss the first case. The other two cases can be handled similarly. Assume that $y_r = v_k$. If there exist $v_i, v_{i+2} \in X$ with $i + 2 < k$ and $w(v_i) < w(y_o) < w(v_{i+2})$, then $\mathcal{P}' = \mathcal{S}_{\mathcal{P}}(v_i^+, y_o)$ contains an increasing weight subsequence S' which is the same as S except that y_o is in S' . It can be found that $\ell_{\mathcal{P}'}(S') = \ell_{\mathcal{P}}(S) + 1$, a contradiction. Assume that $S = \langle x_{\gamma_1}, x_{\gamma_2}, \dots, x_{\gamma_h}, y_r, \dots \rangle$. Thus, the possible cases are either $w(y_o) < w(x_{\gamma_1})$ and $x_{\gamma_1} = v_1$, or $w(x_{\gamma_h}) < w(y_o) < w(y_r)$ and $y_r^- = x_{\gamma_h}$. If $w(y_o) < x_{\gamma_1}$ and $x_{\gamma_1} = v_1$, then $\mathcal{P}' = M_{\mathcal{P}}(y_o^-, v_1)$ contains an increasing weight subsequence S' which is the same as S except that y_o^- is removed from S and y_o is added to the beginning of S . Note that y_o^- must be in S ; otherwise, $\ell_{\mathcal{P}'}(S') = \ell_{\mathcal{P}}(S) + 1$, a contradiction. For the case where $w(x_{\gamma_h}) < w(y_o) < w(y_r)$ and $y_r^- = x_{\gamma_h}$, $\mathcal{P}' = M_{\mathcal{P}}(y_o y_o^+, y_r)$ contains an increasing weight subsequence S' which is the same as S except that y_o^+ is removed from S and y_o is added to in between of x_{γ_h} and y_r . Note that y_o^+ must be in S ; otherwise, $\ell_{\mathcal{P}'}(S') = \ell_{\mathcal{P}}(S) + 1$, a contradiction. This completes the proof of this lemma. $\square$

Now, we are at a position to describe our algorithm.

Algorithm A

Input: A complete bipartite graph $K_{m,n}$ with distinct $w(v)$ for each vertex v in $K_{m,n}$.

Output: A longest increasing subsequence S of $K_{m,n}$.

```

1 begin
2   Sort all vertices in  $X$  and  $Y$ , respectively,
   according to their weights;
3   Assume that  $w(x_1) < w(x_2) < \dots < w(x_m)$ 
   and  $w(y_1) < w(y_2) < \dots < w(y_n)$ ;
4   Let  $S = \langle y_1, y_2, \dots, y_n \rangle$ ;
5   Find  $x_j$  and  $x_k$  with  $w(x_j) < w(y_1)$  and
    $w(y_n) < w(x_k)$ , respectively;
6   if  $m > n$  then
7     insert  $x_j$  and  $x_k$ , if exist, into  $S$  before  $y_1$ 
     and after  $y_n$ , respectively, i.e.,
      $S = \langle x_j, y_1, y_2, \dots, y_n, x_k \rangle$ ;
8   else if  $x_j$  exists then
9     insert  $x_j$  into  $S$  before  $y_1$ ;
10  else if  $x_k$  exists then
11    insert  $x_k$  into  $S$  after  $y_n$ ;
12  end
13  for  $i = 1$  to  $n - 1$  do
14    find an  $x_j$  such that
     $w(y_i) < w(x_j) < w(y_{i+1})$ ;
15    insert  $x_j$ , if it exists, into  $S$  in the place
    between  $y_i$  and  $y_{i+1}$ ;
16  end
17  return  $S$ ;
18 end

```

We use Figure 1 as an example to illustrate Algorithm A, where $X = \{x_1, x_2, x_3, x_4\}$, $Y = \{y_1, y_2, y_3, y_4\}$, $w(x_1) = 1$, $w(x_2) = 4$, $w(x_3) = 6$, $w(x_4) = 8$, $w(y_1) = 2$, $w(y_2) = 3$, $w(y_3) = 5$, and $w(y_4) = 7$. In line 2, the vertices in X and Y are sorted, respectively, according to their weights. In line 4, $S = \langle y_1, y_2, y_3, y_4 \rangle$. In line 5, we can find that $w(x_1) < w(y_1)$ and $w(y_4) < w(x_4)$. Since $m = n$ in this example, lines 8 and 9 are executed, and S becomes $\{x_1, y_1, y_2, y_3, y_4\}$. In lines 13-16, x_2 and x_3 satisfy with $w(y_2) < w(x_2) < w(y_3)$ and $w(y_3) < w(x_3) < w(y_4)$. Thus, S becomes $\{x_1, y_1, y_2, x_2, y_3, x_3, y_4\}$. Note that its corresponding increasing weight subsequence is $\{1, 2, 3, 4, 5, 6, 7\}$, and its corresponding path is $x_1, y_1, x_4, y_2, x_2, y_3, x_3, y_4$.

Theorem 2. For a complete bipartite graph $K_{m,n}$ with distinct vertex weights, Algorithm A finds an LIS in $K_{m,n}$ in $O(|V| \log |V|)$ time.

Proof. By Lemma 1, S contains a subsequence $\langle y_1, y_2, \dots, y_n \rangle$. Since the vertices in X and the vertices in Y appear in alternative in any path in $K_{m,n}$, at most one x_j is in between of y_i and y_{i+1} , for $1 \leq i \leq n-1$. Note that there is also at most one x_i which can be in front of y_1 or behind y_n . Thus, the length of the subsequence found by Algorithm

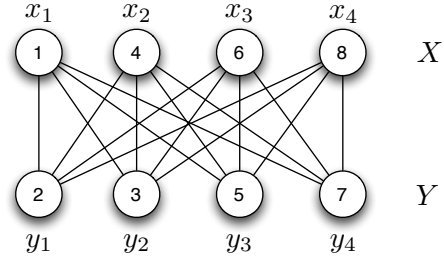


Figure 1: $X = \{x_1, x_2, x_3, x_4\}$ and $Y = \{y_1, y_2, y_3, y_4\}$.

A is equal to $L(G)$.

Now we analyze the time-complexity of Algorithm A. In line 2, it takes $O(|V| \log |V|)$ -time to sort all vertices in X and Y , respectively, according to their weights. Line 5 can be done in constant time by comparing $w(x_1)$ with $w(y_1)$ and $w(x_m)$ with $w(y_n)$. It is clear that lines 6-15 can be done in constant time. The for loop can be done in $O(m+n)$ time by using a technique similar to merging two sorted lists. Finally, line 20 only needs (n) time to report the result. Therefore, the time-complexity of Algorithm A is $O(|V| \log |V|)$. This completes the proof. $\square$

Theorem 3. For a complete bipartite graph $K_{m,n}$, if each vertex v in $K_{m,n}$ has a distinct weight $w(v) \in \{1, 2, \dots, m+n\}$, then $L(K_{m,n})$ can be found in $O(\min\{n \log n, m \log \log m\})$ time.

Proof. Since $w(v) \in \{1, 2, \dots, m+n\}$, the vertices in Y can be sorted in $O(m \log \log m)$ time if $m < n \log n / \log \log m$ [4, 5]; otherwise, in $O(n \log n)$ time. Assume that $w(y_1) < w(y_2) < \dots < w(y_n)$. Then, compute

$$\delta(y_i) = \begin{cases} 0 & \text{if } w(y_i) + 1 = w(y_{i+1}) \\ 1 & \text{otherwise.} \end{cases}$$

for $1 \leq i \leq n-1$. Let $\Delta = \sum_{i=1}^{n-1} \delta(y_i)$. Then, $L(K_{m,n})$ is equal to $n + \Delta$ if $w(y_1) = 1$ and $w(y_n) = m+n$, is equal to $n + \Delta + 1$ if $m = n$ and either $w(y_1) > 1$ or $w(y_n) < m+n$, or exactly one of $w(y_1) > 1$ and $w(y_n) < m+n$ holds true, and is equal to $n + \Delta + 2$ otherwise. Clearly, $L(K_{m,n})$ can be computed in $O(n)$ time. This completes the proof. $\square$

3. Concluding remarks

In this paper, we give a definition for the longest increasing subsequence problem on general graphs. We propose an $O(|V| \log |V|)$ -time algorithm for

solving this problem on complete bipartite $K_{m,n}$. As a future work, it is interesting to study the LIS problem on complete partite graphs with equal weights. Moreover, it is also interesting to study the problem on other graphs.

References

- [1] M.H. Albert, A. Golynski, A.M. Hamel, A. López-Ortiz, S.S. Rao, and M.A. Safari, Longest increasing subsequences in sliding windows, *Theoretical Computer Science* 321(2-3) (2004) 405–414.
- [2] D. Aldous and P. Diaconis, Longest increasing subsequences: from patience sorting to the Baik-Deift-Johansson theorem, *Bulletin of the American Mathematical Society* 36 (1999) 413–432.
- [3] S. Bspamyatnikh and M. Segal, Enumerating longest increasing subsequences and patience sorting, *Information Processing Letters* 76 (2000) 7–11.
- [4] P. Boas, Preserving order in a forest in less than logarithmic time and linear space, *Information Processing Letters* 6 (1977) 80–82.
- [5] P. Boas, R. Kaas, and E. Zijlstra, Design and implementation of an efficient priority queue, *Theory of Computing Systems* 10 (1977) 99–127.
- [6] E. Chen, L. Yang, and H. Yuan, Longest increasing subsequences in windows based on canonical antichain partition, *Theoretical Computer Science* 378(3) (2007) 223–236.
- [7] S.C. Chen, J.Y. Wu, G.S. Huang, and R.C.T. Lee, Finding a longest increasing subsequence on a galled tree, *the 28th Workshop on Combinatorial Mathematics and Computation Theory*, Penghu, Taiwan, 2011.
- [8] M. Crochemore and E. Porat, Fast computation of a longest increasing subsequence and application, *Information and Computation* 208(9) (2010) 1054–1059.
- [9] A. Elmasry, The longest almost-increasing subsequence, *Information Processing Letters* 110(16) (2010) 655–658.
- [10] D.E. Knuth, Permutations, matrices, and generalized Young tableaux, *Pacific Journal of Mathematics* 34 (1970) 709–727.
- [11] M.L. Fredman, On computing the length of longest increasing subsequences, *Discrete Mathematics* 11 (1975) 29–35.
- [12] P. Groeneboom, Hydrodynamical methods for analyzing longest increasing subsequences, *Journal of Computational and Applied Mathematics* 142 (2002) 83–105.
- [13] J. Hunt and T. Szymanski, A fast algorithm for computing longest common subsequences, *Communication of the ACM* 20 (1977) 350–353.
- [14] C. Schensted, Longest increasing and decreasing subsequences, *Canadian Journal of Mathematics* 13 (1961) 179–191.
- [15] I.H. Yang, C.P. Huang, and K.M. Chao, A fast algorithm for computing a longest increasing subsequence, *Information Processing Letters* 93(5) (2005) 249–253.